AuraMart

Event-driven e-commerce platform with an AI shopping assistant

Architecture · Tech stack · Pricing · Technical features · End-to-end features | October 2026

Project overview

AuraMart is a portfolio-grade online store built by one developer as a learning project. Shoppers browse and search products, place orders, and chat with an AI assistant that recommends products from the real catalogue and checks order status. Behind the scenes, separate Spring Boot microservices talk to each other through Apache Kafka events, and checkout uses the Saga pattern so a failed payment cleanly cancels the order and returns the stock.

Learning goals

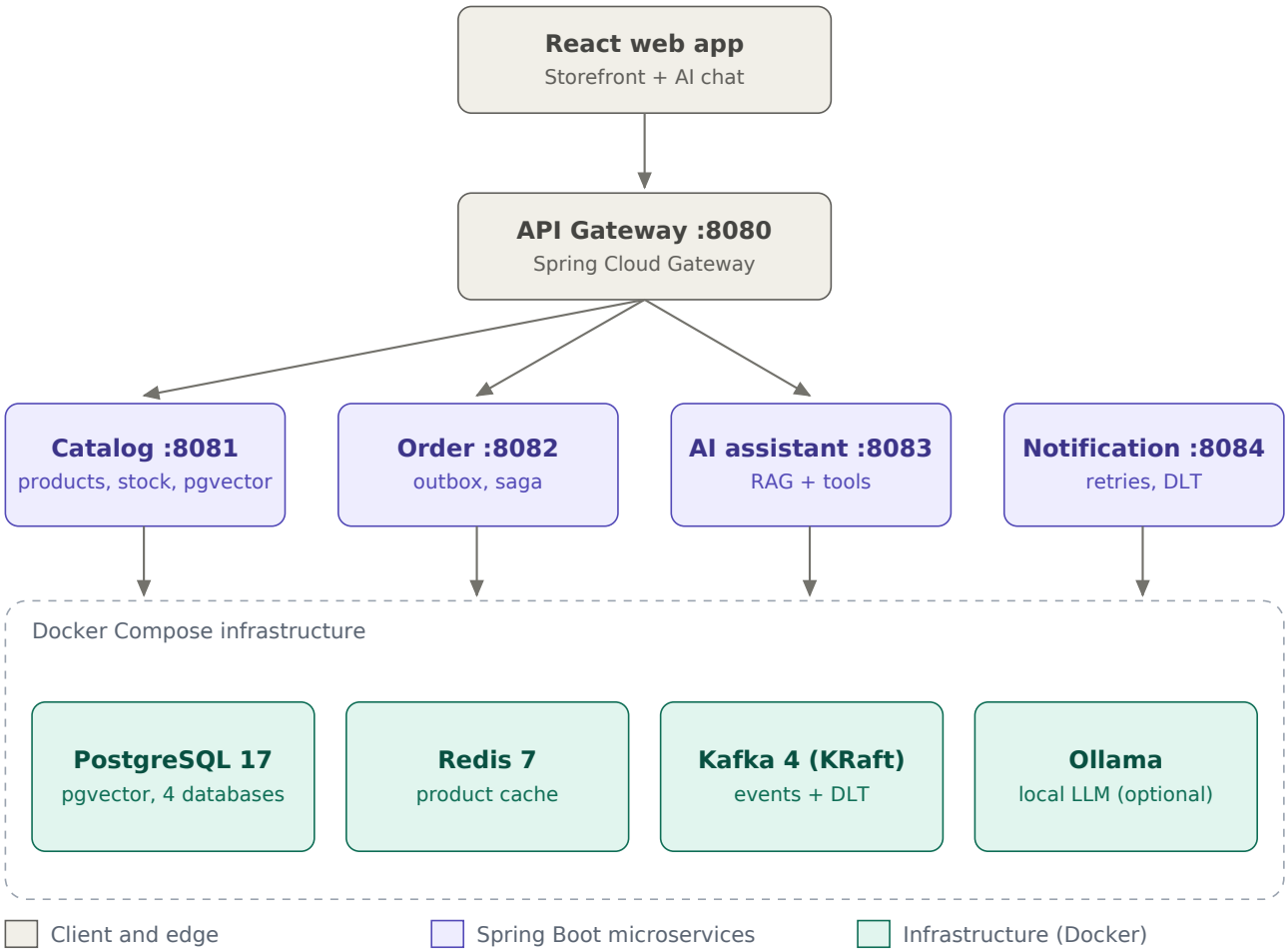
- Microservices with Spring Boot 4.1 and Spring Cloud
- Event-driven architecture with Apache Kafka: producers, consumers, partition keys, Dead Letter Topics, Saga pattern
- AI integration with Spring AI: RAG, pgvector, tool (function) calling, chat memory
- Modern frontend with React and Tailwind CSS
- Production practices: testing, observability, CI and Kubernetes

Contents

- 1. Architecture
- 2. Tech stack
- 3. Pricing
- 4. Technical features
- 5. End-to-end features
- 6. Roadmap at a glance

1. Architecture

The React app talks only to the API Gateway. The gateway routes requests to three services: Catalog, Order and AI assistant. The Notification service has no public API; it only consumes Kafka events. All services share one set of infrastructure containers, but each service owns its own database.



Not drawn above, to keep the picture readable: the AI assistant calls the Catalog service (semantic product search) and the Order service (order status) as LLM tools, and the services exchange events through Kafka as shown in the topic table below.

Services and ports

Component	Port	Responsibility
API Gateway	8080	Single entry point, routing, CORS; JWT security in Phase 6
Catalog service	8081	Products, categories, stock levels, keyword + semantic search, embeddings
Order service	8082	Cart and orders, transactional outbox, saga coordination, simulated payment
AI assistant service	8083	Spring AI chat, RAG, tool calling, chat memory, streaming replies
Notification service	8084	Consumes order events, sends notifications, retries, Dead Letter Topic
PostgreSQL 17 + pgvector	5432	catalog_db, order_db, assistant_db, notification_db (one login each)
Redis 7	6379	Cache for product reads

Component	Port	Responsibility
Kafka 4 (KRaft)	9092	Event bus; no ZooKeeper
Kafka UI	8090	Browse topics, messages, consumer lag and DLT
Ollama (optional)	11434	Free local LLM and embedding model

Kafka topics

Topic	Key	Producer	Consumers	Events
order.events	orderId	Order (via outbox)	Catalog, Notification	OrderPlaced, OrderConfirmed, OrderCancelled
inventory.events	orderId	Catalog	Order	StockReserved, StockRejected
order.events-dlt	orderId	Notification error handler	You (inspect / replay in Kafka UI)	Events that failed after all retries

Using orderId as the key sends every event for one order to the same partition, so they are processed in order, while different orders are processed in parallel across partitions.

Key architecture decisions

Decision	Why
Spring Boot 4.1 instead of 3.x	Spring Boot 3.5 (last 3.x) left free support in June 2026, and Spring AI 2.0 requires Boot 4.0/4.1.
No Eureka service discovery	Docker Compose and Kubernetes already resolve services by name. One less server to run as a solo developer.
One Postgres server, one database + login per service	Enforces database-per-service with permissions while saving laptop RAM.
Stock lives in the Catalog service	Avoids a separate inventory service; the saga reserves and releases stock there.
Embeddings live in the Catalog service	The data owner also owns its search. The AI service calls Catalog through a tool instead of reading its tables.
Choreography saga + transactional outbox	No central orchestrator to build; the outbox guarantees events are published if the database commit succeeds.
Compose runs only infrastructure (Phases 1–5)	Spring services run from the IDE for fast restarts and debugging; containerised in Phase 6.

2. Tech stack

Backend core

Technology	Used for	Phase
Java 21 (LTS)	Language: records, virtual threads, pattern matching	1
Spring Boot 4.1	Base framework for every microservice	1
Spring Web MVC	REST APIs	1
Spring Data JPA + Hibernate	Database access	1
Bean Validation (Jakarta)	Request validation	1
Spring Boot Actuator	Health checks, metrics	1
springdoc-openapi	Swagger UI / API docs	1
Maven (multi-module)	Build, one parent POM for all services	1

Microservices and gateway

Technology	Used for	Phase
Spring Cloud 2025.1	Microservice tooling	2
Spring Cloud Gateway	Single entry point, routing, CORS	2
Docker / Kubernetes DNS	Service discovery (instead of Eureka)	2 / 6
Resilience4j	Circuit breakers, retries, timeouts	6

Data and caching

Technology	Used for	Phase
PostgreSQL 17	Main database, one database per service	1
Flyway	Versioned schema migrations	1
pgvector	Stores product embeddings for semantic search	4
Redis 7 + Spring Cache	Caching product reads	2

Event-driven messaging

Technology	Used for	Phase
Apache Kafka 4 (KRaft, no ZooKeeper)	Event bus between services	3
Spring for Apache Kafka	Producers, consumers, retries	3
Partition keys (orderId)	Keeps events for one order in order	3
Dead Letter Topic (DLT)	Holds events that fail after retries	3
Saga pattern (choreography)	Checkout across order, stock, payment	3
Transactional outbox	Never lose an event if Kafka is down	3
Kafka UI (kafbat)	Browse topics, messages, consumer lag	1

AI

Technology	Used for	Phase
Spring AI 2.0	ChatClient, advisors, AI integration	4
RAG (Retrieval-Augmented Generation)	Answer product questions from your catalog	4
Tool / function calling (@Tool)	AI looks up products and order status	4
PgVectorStore	Spring AI's pgvector integration	4
Chat memory (JDBC)	Remembers the conversation	4
Ollama (free) or OpenAI / Anthropic / Gemini	The LLM and embedding model	4

Frontend

Technology	Used for	Phase
React + Vite	Storefront and AI chat UI	5
TypeScript	Type-safe frontend code	5
Tailwind CSS	Styling	5
TanStack Query	API calls and caching in the browser	5
Server-Sent Events (SSE)	Streaming AI replies word by word	5

Testing

Technology	Used for	Phase
JUnit 5, Mockito, AssertJ	Unit tests	1
Testcontainers	Integration tests against real Postgres, Kafka, Redis	2

DevOps and observability

Technology	Used for	Phase
Docker + Docker Compose	Local infrastructure	1
OpenTelemetry	Distributed tracing across services	6
Prometheus + Grafana	Metrics and dashboards	6
Keycloak (OAuth2 / JWT)	Login and API security	6
GitHub Actions	CI: build and test on every push	6
Kubernetes (kind)	Deployment, Pods, ConfigMaps, autoscaling	6

3. Pricing

The project can be completed for **₹0**. Every tool in the stack is free and open source. Money only comes in for two optional things: a hosted AI model, and putting a live demo online.

Item	Cost
Java, Spring Boot, Spring AI, Kafka, PostgreSQL, Redis, React, Docker	Free (open source)
IntelliJ Community / VS Code, Kubernetes locally (kind)	Free
GitHub repo + GitHub Actions CI	Free for public repos
LLM via Ollama on your laptop	Free (needs RAM)
LLM via hosted API (optional)	**About ₹50–₹500 for the whole project**
Live demo on AWS (optional)	**Free for up to 6 months with credits**
Custom domain name (optional)	Around ₹500–1,000 per year

Why the AI cost is so small

Hosted models are priced per million tokens and a learning project uses very few. Listed OpenAI prices (as of 14 Oct 2026): GPT-5 Nano $0.05 input / $0.40 output, GPT-5 Mini $0.25 / $2.00, and text-embedding-3-small $0.02 per million input tokens.

Estimate (about 2,000 test chats)	Tokens	GPT-5 Nano	GPT-5 Mini
Chat input	~6 million	$0.30	$1.50
Chat output	~1 million	$0.40	$2.00
Total		**~$0.70 (≈ ₹60)**	**~$3.50 (≈ ₹300)**
Embedding the whole catalogue	< 1 million	< ₹1	< ₹1

Gemini also offers a free API tier with limited model access; content sent on it may be used to improve Google's products, so use it only with test data. Prices checked 15 Oct 2026 (sources: modelcompare.dev, aws.amazon.com/free, costbench.com); rupee figures are rounded. Re-check before paying for anything.

AWS for a live demo

- A new AWS account gets $100 in credits immediately, plus up to $100 more for exploring key services. The free plan ends after 6 months or when credits run out, whichever comes first; you are not charged unless you upgrade.
- Cheapest setup: one EC2 instance running the Docker Compose stack.
- Avoid managed Kubernetes (EKS) and managed Kafka (MSK) for this project: they use up credits very fast. Practise Kubernetes locally with kind.

Bottom line

Scenario	Total cost
Ollama (local LLM), no live demo	₹0
Hosted LLM API, no live demo	Under ₹500
Hosted LLM + live demo on AWS credits + domain	Under ₹1,500

Safety tip: set a monthly spending limit on any API account and create an AWS budget alert before you start, so a bug that sends requests in a loop can't cause a surprise bill.

4. Technical features

The engineering features that make AuraMart a strong CV and interview project.

Feature	What it shows you can do	Service(s)	Phase
Microservices, one database each	Split a system by business area	All services	1
API Gateway as single entry point	Routing, CORS, security at the edge	API Gateway	2
Kafka events with orderId as partition key	Keep events in order, process in parallel	Order, Kafka	3
Saga with compensation	Distributed transactions without 2-phase commit	Order, Catalog	3
Transactional outbox	Never lose an event when the database commits	Order	3
Retries with backoff + Dead Letter Topic	Handle failures without blocking the queue	Notification	3
Idempotent consumers	Safely handle duplicate messages	Notification, Catalog	3
Optimistic locking on stock	Prevent overselling under concurrent orders	Catalog	1
Redis cache with eviction	Faster reads, correct invalidation	Catalog	2
RAG with pgvector	AI grounded in your own data	AI assistant, Catalog	4
LLM tool calling	AI that takes real actions through APIs	AI assistant	4
Standard error responses (RFC 9457)	Clean, consistent API design	All services	1
Flyway migrations	Safe, versioned schema changes	All services	1
Testcontainers integration tests	Testing against real infrastructure	All services	2
Distributed tracing with OpenTelemetry	Follow one request across services	All services	6
Kubernetes deployment with autoscaling	Run and scale it like production	All services	6
Architecture Decision Records	Explain trade-offs the way senior engineers do	docs/adr	1

5. End-to-end features

What shoppers and admins see

Area	Feature	Description / example	Service	Phase
Shopping	Browse products	Browse by category with paging and sorting by name or price	Catalog	1
Shopping	Keyword search	Typo-tolerant: "hedphones" still finds headphones	Catalog	2
Shopping	Semantic search	"Something to keep tea hot on a train" finds the steel flask	Catalog (pgvector)	4
Shopping	Stock status	Product pages show price and in-stock / out-of-stock	Catalog	1
Cart & checkout	Cart and ordering	Add to cart, change quantities, place an order	Order	3
Cart & checkout	Checkout flow	Reserves stock, runs a simulated payment, then confirms	Order, Catalog	3
Cart & checkout	Clean cancellation	Failed payment or no stock: order cancelled, stock returned	Order, Catalog	3
Cart & checkout	Order tracking	Order history and status: Pending, Confirmed, Cancelled	Order	3
AI assistant	Natural-language shopping	"Suggest earbuds under ₹5,000 for the gym"	AI assistant	4
AI assistant	Grounded answers (RAG)	Answers only from the real catalogue; doesn't invent products	AI, Catalog	4
AI assistant	Order lookup (tool calling)	"Where is my order?" calls the Order service	AI, Order	4
AI assistant	Conversation memory	"Show me a cheaper one" understands what "one" means	AI assistant	4
AI assistant	Streaming replies	Replies appear word by word	AI, Frontend	5
Notifications	Order notifications	Confirmation and cancellation messages, logged or emailed	Notification	3
Admin	Product management	Create, update, deactivate products via API or Swagger UI	Catalog	1
Admin	Kafka monitoring	Topics, messages and failed (DLT) events in Kafka UI	Infrastructure	3
Admin	Health and tracing dashboards	Service health, request rates and traces in Grafana	All services	6

Flow A — Product discovery with the AI assistant

Step	What happens	Where
1	Shopper types: "I need something to keep tea hot on a long train trip"	React → Gateway → AI assistant
2	Spring AI sends the question and the tool list to the LLM	AI assistant → LLM
3	LLM decides to call the searchProducts tool	LLM → AI assistant
4	Tool runs semantic search: question → embedding → nearest products in pgvector	AI assistant → Catalog
5	Matching products (with price and stock) are returned to the LLM as context	Catalog → AI assistant → LLM
6	LLM writes the answer using only those products; reply streams word by word	AI assistant → React (SSE)

Flow B — Checkout (happy path)

Step	What happens	Where
1	Shopper clicks Place order; order saved as PENDING and OrderPlaced written to the outbox table in the same DB transaction	Order (order_db)
2	Outbox relay publishes OrderPlaced to order.events (key = orderId)	Order → Kafka
3	Catalog consumes OrderPlaced and reserves stock (optimistic locking prevents overselling)	Kafka → Catalog
4	Catalog publishes StockReserved to inventory.events	Catalog → Kafka
5	Order consumes StockReserved and runs the simulated payment	Kafka → Order
6	Payment succeeds: order set to CONFIRMED, OrderConfirmed published	Order → Kafka
7	Notification consumes OrderConfirmed and sends the confirmation	Kafka → Notification

Flow C — Checkout failure and compensation

Step	What happens	Where
1a	Not enough stock: Catalog publishes StockRejected; Order marks the order CANCELLED	Catalog → Kafka → Order
1b	Payment fails after stock was reserved: Order publishes OrderCancelled	Order → Kafka
2	Catalog consumes OrderCancelled and releases the reserved stock (compensation)	Kafka → Catalog
3	Notification sends a cancellation message	Kafka → Notification
4	Every consumer records processed event IDs, so a duplicate event is ignored (idempotency)	All consumers

Flow D — Consumer failure, retries and Dead Letter Topic

Step	What happens	Where
1	Notification fails to send (e.g. email server down)	Notification
2	Spring Kafka retries with exponential backoff (e.g. 1 s, 2 s, 4 s)	Notification
3	Still failing: the event is published to order.events-dlt with error details in headers	Notification → Kafka
4	The main topic keeps flowing; other orders are not blocked	Kafka
5	You inspect the failed event in Kafka UI and replay it after fixing the cause	Kafka UI

Interview story: one scenario that shows every major skill

Step	What happens	Service(s)	Skill shown
1	User asks the AI assistant for a laptop bag	AI assistant	Natural-language chat
2	Assistant finds one through semantic search	AI assistant, Catalog	RAG, pgvector, tool calling
3	User orders it; the order event goes through Kafka	Order, Kafka	Outbox, partition keys
4	Stock is reserved with optimistic locking	Catalog	Optimistic locking, saga
5	Payment fails, so the saga cancels the order and returns the stock	Order, Catalog	Compensation
6	Notification fails twice, retries, and the message ends up in the DLT	Notification	Retries, backoff, DLT
7	Show the whole path in one trace	All services	OpenTelemetry tracing

> "*A user asks the AI assistant for a laptop bag. It finds one through semantic search and the user orders it. The order event goes through Kafka. Stock is reserved with optimistic locking. Payment fails, so the saga cancels the order and returns the stock. The notification consumer fails twice, retries, and the message ends up in the DLT. I can show the whole path in one trace.*"

6. Roadmap at a glance

Phase	Goal	User features	Technical features	Total
1. Foundation	Infra in Compose; Catalog CRUD on Postgres	3	5	8
2. Edge & speed	API Gateway; Redis caching; Testcontainers tests	1	3	4
3. Event-driven checkout	Order service, outbox, Kafka saga, notification consumer, DLT	6	5	11
4. AI assistant	Embeddings in pgvector; RAG; tool calling; chat memory	5	2	7
5. Frontend	Storefront, cart, checkout, streaming chat panel	1	0	1
6. Production polish	Containerise all, tracing, metrics, resilience, auth, CI, Kubernetes	1	2	3
Total		**17**	**17**	**34**